

MECHATRONICS AND IOT

ASSIGNMENT REPORT - 12

TITLE:

Design and develop a GPS-Based Vehicle Tracking and Fleet Management System. The system enables real-time vehicle tracking and efficient fleet monitoring for transportation management.

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The GPS-Based Vehicle Tracking and Fleet Management System is an IoT-based system designed to provide real-time tracking and monitoring of vehicles. The system uses a GPS module to determine the vehicle's current location, speed and movement, while a microcontroller processes the collected data.

The vehicle's location information can be transmitted wirelessly using GSM or Wi-Fi connectivity to a cloud platform or fleet management dashboard. Fleet operators can monitor the vehicle's real-time location, route, speed and travel status remotely.

The system can also maintain location records for route analysis and help identify unauthorized movement, excessive speed and inefficient routes. Alerts can be generated when predefined conditions are detected.

The proposed system provides a low-cost, real-time and efficient solution for vehicle tracking and fleet management, helping improve transportation safety, vehicle utilization, route planning and overall fleet operational efficiency.

Problem Statement:

Managing a fleet of vehicles efficiently requires continuous monitoring of vehicle location, speed, route and movement status. Traditional fleet management methods often depend on manual tracking and communication, making it difficult to obtain accurate real-time information about vehicles.

Lack of real-time tracking can lead to route inefficiency, unauthorized vehicle usage, excessive travel time, fuel wastage and delayed response during emergencies. Fleet operators may also find it difficult to monitor multiple vehicles simultaneously and analyze their travel patterns.

Therefore, there is a need for a real-time GPS-based vehicle tracking system that can continuously determine vehicle location, transmit tracking information remotely and provide useful fleet data for monitoring, route analysis and transportation management.

Proposed Solution:

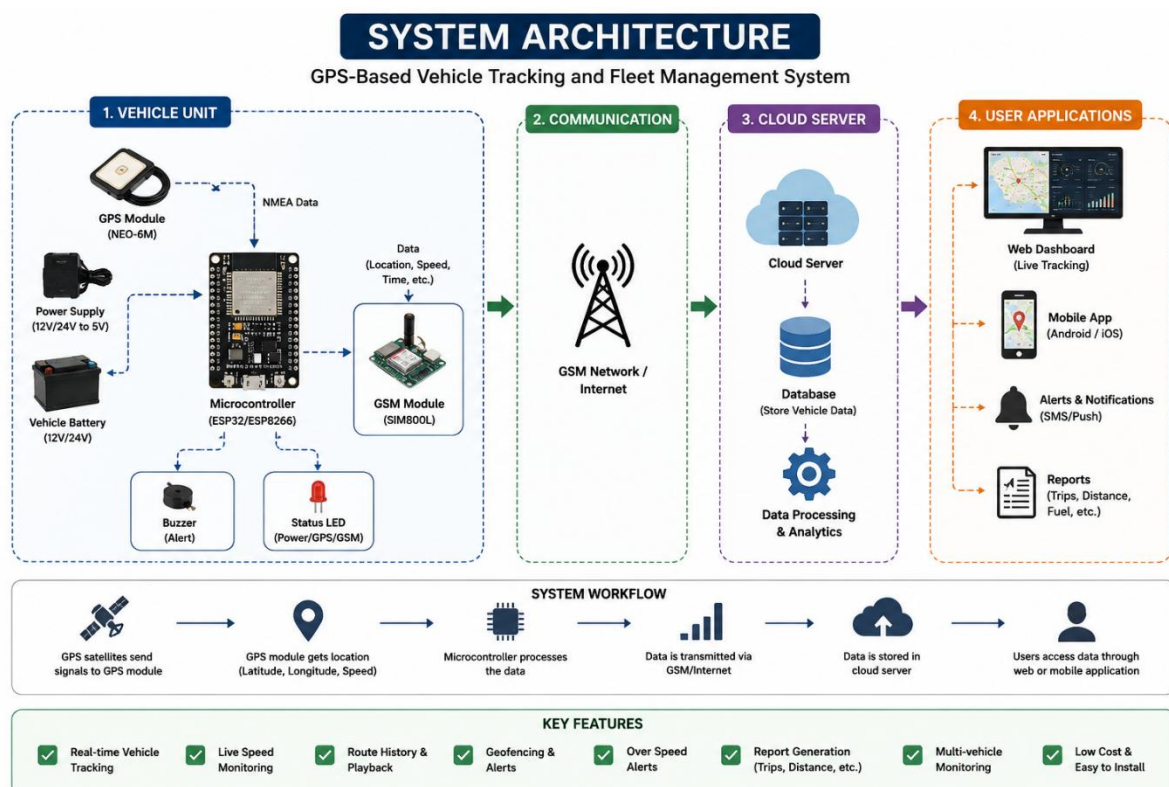
The proposed system uses a GPS module and ESP8266/ESP32 microcontroller to continuously determine the vehicle's location, speed and movement status. The GPS module receives signals from satellites and provides the vehicle's geographical coordinates, which are processed by the microcontroller.

The collected location data can be transmitted through a GSM or Wi-Fi communication module to a cloud platform or fleet management dashboard. Fleet operators can remotely monitor the vehicle's real-time location, speed and route using the transmitted data.

The system can compare the vehicle speed with a predefined limit and generate an alert when excessive speed or unauthorized movement is detected. Location data can also be stored for route history and travel analysis.

The proposed solution provides a low-cost, real-time and automated vehicle tracking system that helps improve fleet visibility, transportation safety, route management and overall vehicle utilization.

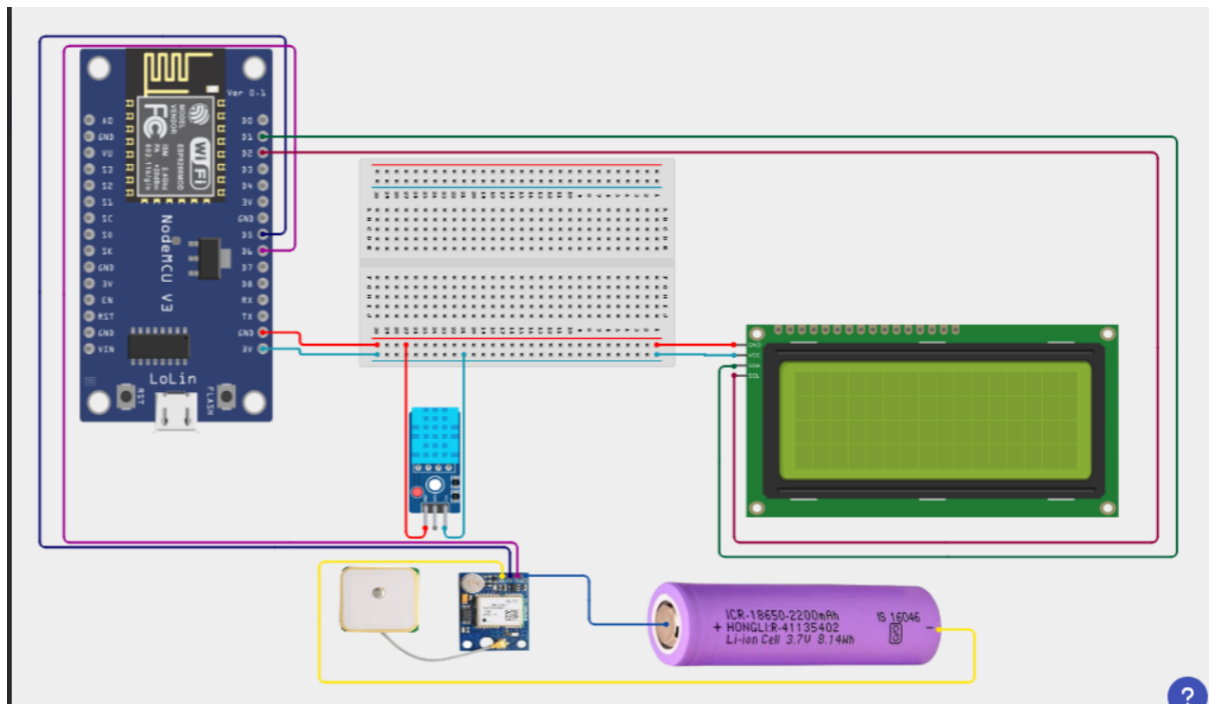
System Architecture:



Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
GPS Module (NEO-6M)	VCC	3.3V	Power supply
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication
GSM Module (SIM800L)	Positive (+)	External 4V supply	Power supply
Green LED	Anode (+)	D5 (GPIO 14) through 220 Ω	Normal condition
Battery	Positive (+)	Voltage regulator $\rightarrow$ ESP32 VIN	Main power supply

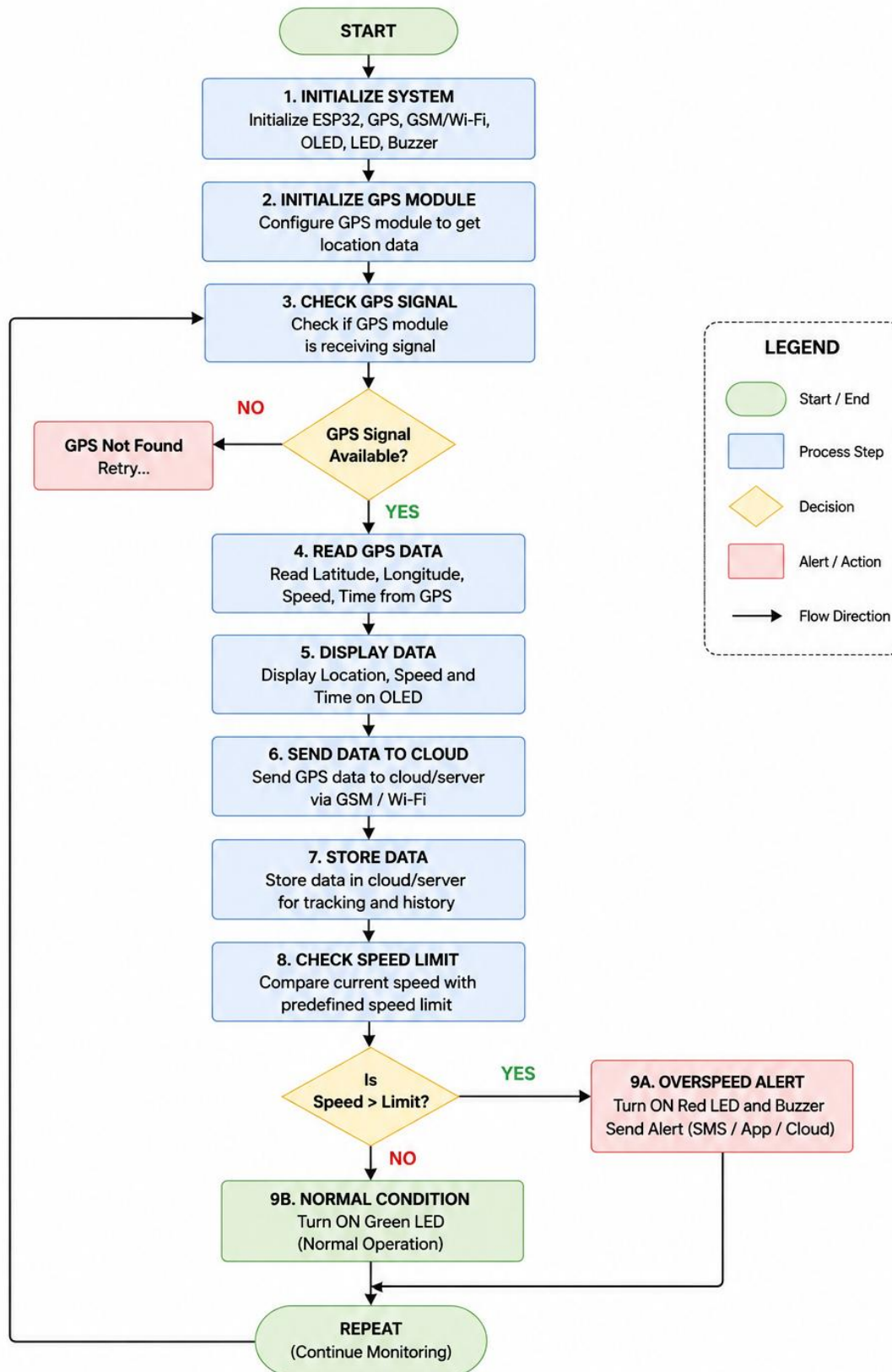
Circuit Design:



Algorithm:

ALGORITHM

GPS-Based Vehicle Tracking and Fleet Management System



Coding:

```
#include <ESP8266WiFi.h>

#include <TinyGPS++.h>

#include <SoftwareSerial.h>

#include <DHT.h>

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"

// =====

// WIFI

// =====

#define WIFI_SSID    "Hari"

#define WIFI_PASSWORD "11111111"

// =====

// ADAFRUIT IO

// =====

#define AIO_SERVER    "io.adafruit.com"

#define AIO_SERVERPORT 1883

#define AIO_USERNAME  "Venkyb_123"

#define AIO_KEY       "aio_conN37l9SluXBamYgS80MFhrRapM"

// =====

// DHT11

// =====

#define DHTPIN 13    // D7

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// =====

// MQ-2

// Digital output only

// =====
```

```
#define MQ2_DO 0    // D3

// Most MQ-2 modules:

// LOW = gas detected

// HIGH = normal

// =====

// =====

// PULSE SENSOR

// =====

#define PULSE_PIN A0

// =====

// BUZZER

// =====

#define BUZZER_PIN 16 // D0

// =====

// GPS

//

// GPS TX -> ESP8266 D5

// GPS RX -> ESP8266 D6

// =====

#define GPS_RX 14    // D5

#define GPS_TX 12    // D6

TinyGPSPlus gps;

SoftwareSerial gpsSerial(

    GPS_RX,

    GPS_TX

);

// =====

// WIFI CLIENT

// =====

WiFiClient client;
```

```
// =====  
// MQTT  
// =====  
Adafruit_MQTT_Client mqtt(  
    &client,  
    AIO_SERVER,  
    AIO_SERVERPORT,  
    AIO_USERNAME,  
    AIO_KEY  
);  
// =====  
// ACTIVE ADAFRUIT FEEDS  
// =====  
// Gas  
Adafruit_MQTT_Publish gasFeed =  
    Adafruit_MQTT_Publish(  
        &mqtt,  
        AIO_USERNAME "/feeds/gas"  
    );  
// Heart rate  
Adafruit_MQTT_Publish heartRateFeed =  
    Adafruit_MQTT_Publish(  
        &mqtt,  
        AIO_USERNAME "/feeds/heart-rate"  
    );  
// Heart rate graph  
Adafruit_MQTT_Publish heartRateGraphFeed =  
    Adafruit_MQTT_Publish(  
        &mqtt,  
        AIO_USERNAME "/feeds/heart-rate-graph"
```



```
);  
  
// Temperature  
Adafruit_MQTT_Publish temperatureFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/temperature"  
);  
  
// Humidity  
Adafruit_MQTT_Publish humidityFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/humidity"  
);  
  
// Worker status  
Adafruit_MQTT_Publish workerStatusFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/worker-status"  
);  
  
// Altitude  
Adafruit_MQTT_Publish altitudeFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/altitude"  
);  
  
// GPS Map  
Adafruit_MQTT_Publish gpsMapFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/gps-map"
```

```
);  
  
// =====  
  
// SENSOR VARIABLES  
  
// =====  
  
float temperature = 0;  
float humidity = 0;  
int gasState = 0;  
int heartRate = 0;  
float altitude = 0;  
double latitude = 0;  
double longitude = 0;  
int gpsSatellites = 0;  
bool gpsFix = false;  
  
// =====  
  
// TIMERS  
  
// =====  
  
unsigned long lastUpload = 0;  
unsigned long lastDHTRead = 0;  
unsigned long lastGPSPrint = 0;  
const unsigned long UPLOAD_INTERVAL = 30000;  
const unsigned long DHT_INTERVAL = 2000;  
  
// =====  
  
// HEART RATE VARIABLES  
  
// =====  
  
int pulseThreshold = 550;  
bool pulseDetected = false;  
unsigned long lastBeatTime = 0;  
int beatCount = 0;  
unsigned long beatTimes[10];  
int beatIndex = 0;
```

```
// =====  
// WIFI  
// =====  
  
void connectWiFi()  
{  
  if (WiFi.status() == WL_CONNECTED)  
    return;  
  
  Serial.println();  
  Serial.println("Connecting to WiFi...");  
  WiFi.mode(WIFI_STA);  
  WiFi.begin(  
    WIFI_SSID,  
    WIFI_PASSWORD  
  );  
  unsigned long start = millis();  
  while (  
    WiFi.status() != WL_CONNECTED &&  
    millis() - start < 15000  
  )  
  {  
    delay(500);  
    Serial.print(".");  
  }  
  Serial.println();  
  if (WiFi.status() == WL_CONNECTED)  
  {  
    Serial.println("WIFI CONNECTED");  
    Serial.print("IP: ");  
    Serial.println(WiFi.localIP());  
  }  
}
```

```

else

{
    Serial.println("WIFI CONNECTION FAILED");
}
}

// =====

// MQTT

// =====

bool connectMQTT()
{
    if (mqtt.connected())
        return true;
    if (WiFi.status() != WL_CONNECTED)
    {
        connectWiFi();
        if (WiFi.status() != WL_CONNECTED)
            return false;
    }
    Serial.println();
    Serial.println("Connecting to Adafruit IO...");
    int8_t ret;
    for (int i = 0; i < 3; i++)
    {
        ret = mqtt.connect();
        if (ret == 0)
        {
            Serial.println("ADAFRUIT IO CONNECTED");
            return true;
        }
    }
    Serial.print("MQTT ERROR: ");

```

```
Serial.println(
    mqtt.connectErrorString(ret)
);
mqtt.disconnect();
delay(2000);
}
return false;
}

// =====
// READ GPS
// =====

void readGPS()
{
    while (gpsSerial.available())
    {
        char c = gpsSerial.read();
        gps.encode(c);
    }
    if (gps.location.isValid())
    {
        gpsFix = true;
        latitude =
            gps.location.lat();
        longitude =
            gps.location.lng();
    }
    if (gps.altitude.isValid())
    {
        altitude =
            gps.altitude.meters();
    }
}
```

```

}
if (gps.satellites.isValid())
{
    gpsSatellites =
        gps.satellites.value();
}
}

// =====

// READ DHT11

// =====

void readDHT()
{
    if (
        millis() - lastDHTRead <
        DHT_INTERVAL
    )
    {
        return;
    }
    lastDHTRead = millis();
    float newHumidity =
        dht.readHumidity();
    float newTemperature =
        dht.readTemperature();
    if (
        isnan(newHumidity) ||
        isnan(newTemperature)
    )
    {
        Serial.println("DHT11 READ FAILED");
    }
}

```

```
    return;
}
humidity =
    newHumidity;
temperature =
    newTemperature;
}
// =====
// READ MQ-2
// =====
void readGas()
{
    int mq2Value =
        digitalRead(MQ2_DO);
    if (mq2Value == LOW)
    {
        gasState = 1;
    }
    else
    {
        gasState = 0;
    }
}
// =====
// READ PULSE SENSOR
// =====
void readPulse()
{
    int signal =
        analogRead(PULSE_PIN);
```

```
unsigned long now =  
    millis();  
// Detect rising pulse  
if (  
    signal > pulseThreshold &&  
    !pulseDetected  
)  
{  
    pulseDetected = true;  
    if (lastBeatTime > 0)  
    {  
        unsigned long interval =  
            now - lastBeatTime;  
        // Accept realistic heartbeat intervals  
        if (  
            interval >= 300 &&  
            interval <= 2000  
        )  
        {  
            int bpm =  
                60000 / interval;  
  
            // Store beat time  
            beatTimes[beatIndex] =  
                now;  
            beatIndex++;  
            if (beatIndex >= 10)  
                beatIndex = 0;  
            beatCount++;  
            // Simple averaging
```



```

int validCount = 0;
unsigned long total =
    0;
for (int i = 0; i < 10; i++)
{
    if (
        beatTimes[i] > 0 &&
        now - beatTimes[i] < 15000
    )
    {
        validCount++;
    }
}
// Use current BPM initially
if (
    bpm >= 40 &&
    bpm <= 180
)
{
    heartRate = bpm;
}
}
}
lastBeatTime = now;
}
// Wait until signal falls
if (
    signal < pulseThreshold - 50
)
{

```

```

    pulseDetected = false;
}
}
// =====
// WORKER STATUS
// =====
String getWorkerStatus()
{
    // Gas alert
    if (gasState == 1)
    {
        return "ALERT";
    }
    // Heart rate abnormal
    if (
        heartRate > 0 &&
        (
            heartRate < 50 ||
            heartRate > 120
        )
    )
    {
        return "ALERT";
    }
    return "SAFE";
}
// =====
// BUZZER
// =====
void updateBuzzer()

```

```
{
String status =
    getWorkerStatus();
if (status == "ALERT")
{
    digitalWrite(
        BUZZER_PIN,
        HIGH
    );
}
else
{
    digitalWrite(
        BUZZER_PIN,
        LOW
    );
}
}

// =====
// PRINT SENSOR DATA
// =====

void printSensorData()
{
    Serial.println();
    Serial.println("=====");
    Serial.println(" WORKER HEALTH MONITOR");
    Serial.println("=====");
    Serial.print("Temperature : ");
    Serial.print(temperature, 1);
    Serial.println(" C");
}
```

```
Serial.print("Humidity  : ");
Serial.print(humidity, 1);
Serial.println(" %");
Serial.print("Gas      : ");
if (gasState == 1)
    Serial.println("ALERT");
else
    Serial.println("NORMAL");
Serial.print("Heart Rate : ");
if (heartRate > 0)
{
    Serial.print(heartRate);
    Serial.println(" BPM");
}
else
{
    Serial.println("Waiting...");
}
Serial.print("Status   : ");
Serial.println(
    getWorkerStatus()
);
if (gpsFix)
{
    Serial.print("Latitude  : ");
    Serial.println(
        latitude,
        6
    );
    Serial.print("Longitude  : ");
```

```

Serial.println(
    longitude,
    6
);
Serial.print("Altitude  : ");
Serial.print(
    altitude,
    2
);
Serial.println(" m");
Serial.print("Satellites : ");
Serial.println(
    gpsSatellites
);
}
else
{
    Serial.println("GPS      : NO FIX");
}
Serial.println("=====");
}
// =====
// SEND TO ADAFRUIT IO
// =====

void sendToAdafruit()
{
    if (!connectMQTT())
    {
        Serial.println(
            "MQTT NOT CONNECTED"

```

```
);  
return;  
}  
Serial.println();  
Serial.println(  
    "Uploading sensor data..."  
);  
// =====  
// GAS  
// =====  
if (  
    gasFeed.publish(gasState)  
)  
{  
    Serial.print("Gas -> ");  
    Serial.println(gasState);  
}  
else  
{  
    Serial.println("Gas -> FAILED");  
}  
delay(300);  
// =====  
// HEART RATE  
// =====  
if (heartRate > 0)  
{  
    if (  
        heartRateFeed.publish(  
            heartRate
```

```
)  
)  
{  
  Serial.print("Heart Rate -> ");  
  Serial.println(heartRate);  
}  
delay(300);  
// Heart rate graph  
if (  
  heartRateGraphFeed.publish(  
    heartRate  
  )  
)  
{  
  Serial.print(  
    "Heart Rate Graph -> "  
  );  
  Serial.println(  
    heartRate  
  );  
}  
}  
delay(300);  
// =====  
// TEMPERATURE  
// =====  
if (  
  temperatureFeed.publish(  
    temperature  
  )  
)
```

```
)  
{  
  Serial.print("Temperature -> ");  
  Serial.println(  
    temperature,  
    1  
  );  
}  
delay(300);  
  
// =====  
  
// HUMIDITY  
  
// =====  
  
if (  
  humidityFeed.publish(  
    humidity  
  )  
)  
{  
  Serial.print("Humidity -> ");  
  Serial.println(  
    humidity,  
    1  
  );  
}  
delay(300);  
  
// =====  
  
// WORKER STATUS  
  
// =====  
  
String status =  
  getWorkerStatus();
```



```
if (
  workerStatusFeed.publish(
    status.c_str()
  )
)
{
  Serial.print("Worker Status -> ");
  Serial.println(status);
}
delay(300);

// =====
// ALTITUDE
// =====

if (gpsFix)
{
  if (
    altitudeFeed.publish(
      altitude
    )
  )
  {
    Serial.print("Altitude -> ");
    Serial.println(
      altitude,
      2
    );
  }
  delay(300);

  // =====
  // GPS MAP
```

```

// =====

String gpsData =
  "{"value\:1,\"lat\":" +
  String(latitude, 6) +
  "\",\"lon\":" +
  String(longitude, 6) +
  "\",\"ele\":" +
  String(altitude, 2) +
  "}";
if (
  gpsMapFeed.publish(
    gpsData.c_str()
  )
)
{
  Serial.println(
    "GPS Map -> UPDATED"
  );
}
}
Serial.println();
Serial.println(
  "UPLOAD COMPLETE"
);
}

// =====

// SETUP

// =====

void setup()
{

```

```
Serial.begin(115200);  
delay(1500);  
Serial.println();  
Serial.println(  
    "===== "  
);  
Serial.println(  
    " WORKER HEALTH & SAFETY MONITORING "  
);  
Serial.println(  
    "===== "  
);  
// -----  
// PINS  
// -----  
pinMode(  
    MQ2_DO,  
    INPUT  
);  
pinMode(  
    BUZZER_PIN,  
    OUTPUT  
);  
digitalWrite(  
    BUZZER_PIN,  
    LOW  
);  
// -----  
// DHT  
// -----
```

```
dht.begin();

// -----

// GPS

// -----

gpsSerial.begin(9600);

Serial.println(

  "GPS initialized"

);

Serial.println(

  "GPS TX -> D5"

);

Serial.println(

  "GPS RX -> D6"

);

// -----

// WIFI

// -----

connectWiFi();

Serial.println();

Serial.println(

  "Sensors initialized."

);

Serial.println(

  "Place finger on Pulse Sensor."

);

Serial.println(

  "Waiting for GPS fix..."

);

}

// =====
```

```
// LOOP

// =====

void loop()

{
    // =====

    // GPS
    // =====

    readGPS();

    // =====

    // DHT11
    // =====

    readDHT();

    // =====

    // MQ2
    // =====

    readGas();

    // =====

    // PULSE SENSOR
    // =====

    readPulse();

    // =====

    // BUZZER
    // =====

    updateBuzzer();

    // =====

    // PRINT EVERY 2 SECONDS
    // =====

    if (
        millis() - lastGPSPrint >= 2000
    )
```

```
{
  lastGPSPrint =
    millis();
  printSensorData();
}

// =====

// ADAFRUIT UPLOAD EVERY 30 SECONDS

// =====

if (
  millis() - lastUpload >=
  UPLOAD_INTERVAL
)
{
  lastUpload =
    millis();
  sendToAdafruit();
}

// =====

// MQTT

// =====

if (mqtt.connected())
{
  mqtt.processPackets(10);
  mqtt.ping();
}
delay(10);
}
```

System Working:

- The GPS-Based Vehicle Tracking and Fleet Management System uses a GPS module, ESP32/ESP8266 microcontroller and GSM/Wi-Fi communication to continuously monitor the vehicle's location and movement.
- The GPS module receives signals from satellites and determines the vehicle's latitude, longitude, speed and time. This information is sent to the microcontroller, which processes the GPS data and prepares it for transmission.
- The processed data is transmitted through the GSM network or Wi-Fi to a cloud server. The location and vehicle information can then be viewed through a web dashboard or mobile application, allowing fleet operators to monitor vehicles remotely in real time.
- The system can also compare the vehicle's speed with a predefined limit and generate an alert when overspeeding or unauthorized movement is detected. Location data can be stored in the cloud for route history, distance analysis and fleet performance monitoring.
- The system continuously repeats the tracking process, providing a real-time, automated and efficient solution for vehicle tracking and fleet management.

Bill Of Material:

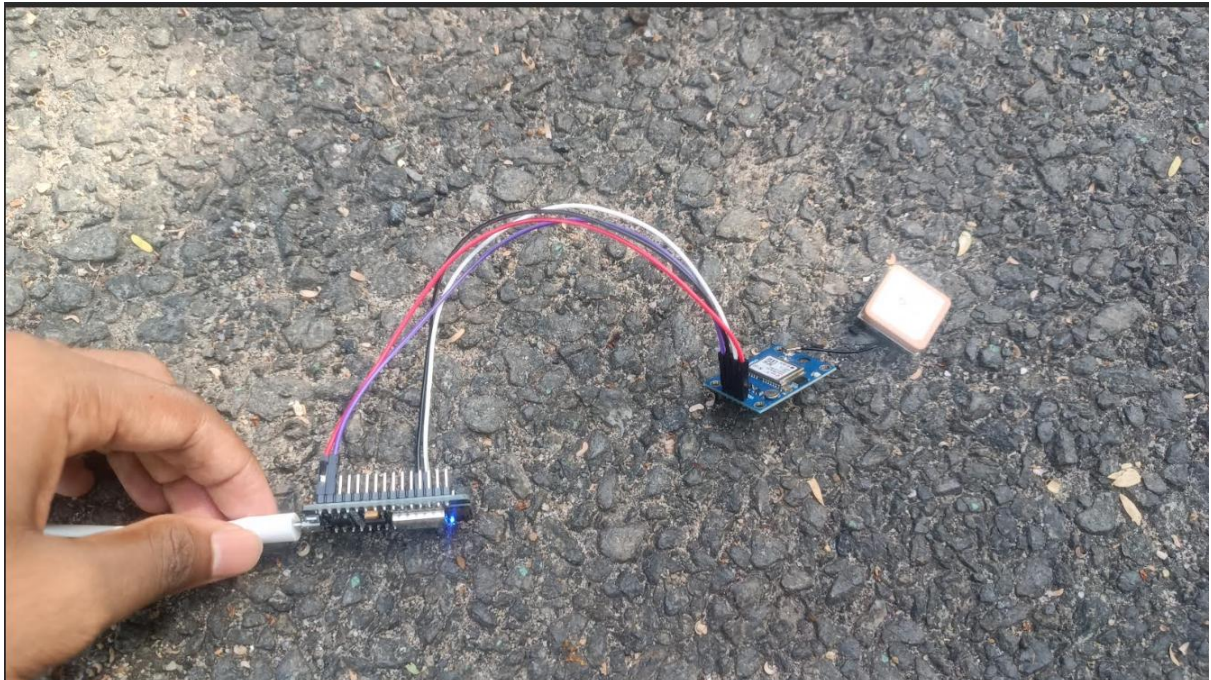
S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	GPS Module	NEO-6M GPS	1	Rs. 350
3	GSM Module	SIM800L	1	Rs. 300
4	Bread Board	Standard 830-point	1	Rs. 100
5	Jumper Wire	Male-Male / Male-Female	1 set	Rs. 50
6	OLED Display	0.96" I ² C, 128×64	1	Rs. 150
		Total		Rs. 1400

Prototype Implementation:

- The prototype was implemented using a GPS module (NEO-6M), ESP32 microcontroller, GSM module, OLED display, LED and buzzer. The GPS module receives satellite signals and provides the vehicle's latitude, longitude, speed and time.
- The ESP32 collects and processes the GPS data and displays the current location and speed on the OLED display. The system checks the vehicle speed against a predefined speed limit and activates the buzzer and red LED when overspeeding is detected.
- The processed GPS information can be transmitted through the GSM module to a cloud server for remote monitoring. Fleet operators can access the vehicle's location and travel information through a web or mobile dashboard.
- The prototype continuously updates the vehicle position and supports real-time tracking, speed monitoring and route analysis.

Prototype flow:

GPS Module → ESP32 → Location & Speed Processing → OLED + LED/Buzzer → GSM/Wi-Fi → Cloud → Fleet Dashboard



Results and Performance:

- The developed GPS-Based Vehicle Tracking and Fleet Management System successfully monitored the vehicle's location, speed and movement status using the GPS module and ESP32. The GPS coordinates were processed in real time and displayed on the OLED display.
- The system successfully detected the vehicle's geographical position and compared the vehicle speed with a predefined limit. When overspeeding was detected, the red LED and buzzer provided an immediate warning. The GPS data could also be transmitted through GSM/Wi-Fi for remote monitoring.

Performance Analysis

Parameter	Result
GPS Location Tracking	Successfully implemented
Latitude & Longitude	Real-time GPS data
Speed Monitoring	Successfully measured
Overspeed Detection	LED + Buzzer alert
Local Display	OLED display
Wireless Communication	GSM/Wi-Fi supported
Remote Monitoring	Cloud integration supported
Route Monitoring	Location data can be stored
Response	Real-time

Overall, the prototype demonstrated a low-cost, real-time and reliable vehicle tracking solution. The system can improve fleet visibility, transportation safety, route management and vehicle utilization, while providing useful data for fleet operators and preventive monitoring.

[Shop](#) [Learn](#) [Blog](#) [Forums](#) [IO](#) [LIVE!](#) [AdaBox](#) [Account](#) [0](#)

 [Devices](#) [Feeds](#) [Dashboards](#) [Actions](#) [Power-Ups](#)

 [New Device](#)

Venkyb_123 / Dashboards / GPS Vehicle Tracking

[+ New Block](#) [Edit Layout](#)  

Latitude

11.270254

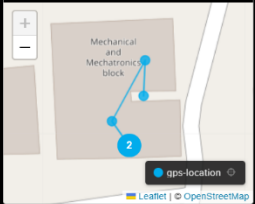
Longitude

77.603986

Satellites

6

Gps-location



Speed



What is Adafruit IO?

IO Status ●

"If opportunity doesn't knock, build a door"

34°C
Partly sunny

  Search



ENG
IN

   36%

02:01 PM
20-08-2026